

2025年度
修士論文

SN開発におけるデバッグシステムの開発

指導教員	アサノ デービッド	教授
	不破 泰	肩書き

2025年1月30日提出

信州大学大学院 総合理工学研究科 工学専攻 情報数理・融合システム分野
24W6047A
辻村篤志

あらまし

あらましを書く。

目次

第1章	はじめに	1
1.1	研究背景	1
1.2	当研究室における先行研究	2
1.3	先行研究の課題	2
1.4	本研究の目的	2
1.5	本論文の構成	2
第2章	先行研究の提案システム	4
2.1	システムの概要	4
2.2	実現事項	5
2.3	課題	5
第3章	提案システムの概要と設計方針	6
3.1	設計方針	6
3.2	システムの構成	7
3.3	各コンポーネントの概要	7
3.3.1	コード生成コンポーネント	8
3.3.2	組込みシステムコンポーネント	8
3.3.3	デバッグ支援システムコンポーネント	9
第4章	設計及び実装	10
4.1	使用ソフトウェア、ツール	10
4.2	各コンポーネントの接続	11
4.3	デバッグシステムの設計	11
4.3.1	動作モード	12

4.4	ログ設計	12
4.4.1	ログ生成の設計	12
4.5	ログ解析の設計	14
4.5.1	ログモードにおけるログ解析	14
4.5.2	同期モードにおけるログ解析	16
4.6	ログモードの設計	16
4.6.1	ログモードの UI 設計	17
4.7	同期モードの設計	17
4.7.1	ブラウザ上でのシリアルモニタ実装	18
4.7.2	同期タイムラインの生成	18
4.8	状態遷移可視化の設計	19
4.9	実装概要	19
第 5 章	操作例	20
5.1	操作例の概要	20
5.2	操作対象および前提条件	20
5.3	状態遷移図の読み込み	21
5.4	ログ取得と状態遷移の可視化	21
5.5	ステップ実行による逐次確認	21
5.6	変数ウォッチ機能の利用	21
5.7	複数ノード同期デバッグ	22
5.8	操作例のまとめ	22
第 6 章	評価と考察	23
6.1	評価方針	23
6.2	評価方法	23
6.3	デバッグ効率の評価	23
6.4	状態遷移理解の容易性に関する評価	24
6.5	複数ノード同期デバッグの評価	24
6.6	考察	24
6.7	評価のまとめ	24

第7章	まとめと今後の展望	25
7.1	まとめ	25
7.2	今後の展望	25
参考文献		26

図 目 次

2.1	先行研究システムの概要図	4
3.1	従来のデバッグ方法	7
3.2	提案システムの概要図	8
3.3	どこでモデム (左) と GPS モジュール (右)	9
4.1	変数更新ロジック	15
4.2	ログモードのレイアウト	17
4.3	同期モードのタイムライン	19
4.4	同期モードのタイムライン	19

表 目 次

第 1 章

はじめに

はじめにを書く。

1.1 研究背景

近年、無線センサネットワーク（Wireless Sensor Network：WSN）や IoT（Internet of Things）は、環境モニタリングやスマートシティ、インフラ監視、農業、ヘルスケアなど、さまざまな分野での応用が期待されている。実際、IoT 全体の接続デバイス数は急速に増加している。IoT Analytics によれば、2023 年時点で約 166 億台、2024 年には約 188 億台に達し、2030 年には約 400 億台に到達すると予測されている [1]。また、WSN の普及も市場規模の面から拡大を続けており、Grand View Research の報告では、産業用 WSN（Industrial WSN）の市場は 2023 年に約 51.9 億ドルと推定され、2024 年から 2030 年にかけて年平均 12.1%の成長が見込まれている [2]。

しかし、これらのシステムを構築するためには、複雑な通信プロトコルの設計やデータ処理の設計および実装が求められ、開発者には幅広い専門知識と多大な工数が必要となる。さらに、無線通信環境の構築やマイコン設定など導入時のハードルも高く、これらの研究や開発の多くはシミュレーション環境やエミュレータ上での検証にとどまることが多い。しかし、シミュレーション環境では実機特有の挙動や外部環境の影響を完全に再現することが難しく、実機での検証が不可欠である場合も多く、実機でのプロトコル検証を容易にするための支援環境の整備が求められている。

1.2 当研究室における先行研究

当研究室の先行研究では、状態遷移図上で無線通信プロトコルの動作を定義し、そこから自動生成したプログラムを汎用ハード上で動作させ、プロトコルを実機で動作検証できる環境が構築されていた（旭ら [3]、小林ら [4]）。

1.3 先行研究の課題

このシステムにより基本的な通信プロトコルの動作検証が可能となったが、デバッグ方法はコンソールへのログ出力に依存しており、通信処理が高速に進むため逐次的な状態遷移を追跡することが難しかった。その結果、複雑な動作を含むプロトコルに対しては、異常動作の原因を把握しづらく、開発効率や教育的利用の観点では十分でない点が課題として残されていた。

1.4 本研究の目的

本研究の目的は、先行研究で課題となっていたデバッグ効率の低さを改善し、実機でのプロトコル検証をより効果的に行える環境を実現することである。そのために、無線通信デバイス上で実行された動作をログとして出力し、それを可視化・ステップ実行できる仕組みを開発した。これにより、プロトコルの動作を逐次的に把握でき、異常動作の原因究明を容易にするとともに、教育やチーム開発、応用実証に適した支援環境を提供することを目指す。

1.5 本論文の構成

本論文は全7章から構成されている。第1章では、本研究の背景として当研究室におけるこれまでの研究の流れを整理し、先行研究で顕在化した課題を明らかにした上で、本研究の目的を述べる。第2章では、当研究室における先行研究システムについて、その構成や実現されてきた機能を整理するとともに、運用およびデバッグの観点から課題をまとめる。第3章では、先行研究の課題を踏まえ、本研究で提案するシステムの概要と設計方針について述べる。ここでは、全体構成および各コンポーネントの役割を中心に説

明する。第4章では、提案システムの設計および実装について述べ、特にデバッグ支援を目的としたログ取得方式や状態遷移の可視化手法について詳述する。第5章では、提案システムの操作例を示し、実際の利用を通してどのような情報が得られるかを説明する。第6章では、従来手法との比較を通じて提案システムの有効性を評価し、その結果について考察を行う。最後に第7章では、本研究のまとめを行うとともに、今後の課題および展望について述べる。

第 2 章

先行研究の提案システム

2.1 システムの概要

先行研究では、無線通信プロトコルの設計・実装・動作検証を支援するシステムが提案されてきた。このシステムは主に次の二つのコンポーネントから構成されている。第一に、UML の状態遷移図を用いてプロトコルの動作を定義し、そこからコードを自動生成するコンポーネントである。第二に、生成されたコードを実行するための環境（ハードウェア）を提供するコンポーネントである。これにより、状態遷移図で定義されたプロトコルが実機上で動作し、その動作を検証できる環境が提供されてきた。システムの概要を図 2.1 に示す。

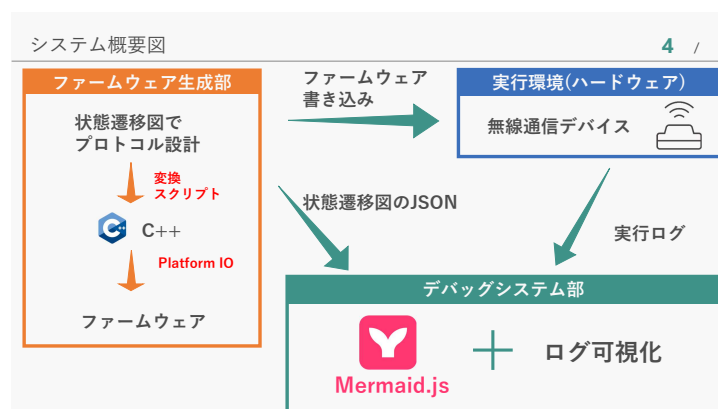


図 2.1: 先行研究システムの概要図

2.2 実現事項

先行研究システムでは、以下の主要な機能が実現されている。

- 状態遷移図からのコード自動生成：UML の状態遷移図を用いてプロトコルの動作を定義し、そこから C 言語コードを自動生成する機能が提供されている。これにより、プロトコル設計者は視覚的に動作を定義でき、手動でのコーディング作業を削減できる。
- 実機上での動作検証：生成されたコードは、無線モデム搭載のマイコンを組み合わせたハードウェア上で実行される。これにより、シミュレーション環境では捉えきれない実機特有の挙動を検証できる。

2.3 課題

先行研究システムには以下の課題が存在する。

- 状態遷移図の記法の制約：先行研究における状態遷移図からのコード自動生成は、特定の記法に依存しており、複雑なプロトコル動作を表現する際に制約がある。これにより、設計者が意図する動作を完全に反映できない場合がある。
- デバッグ効率の低さ：デバッグ方法がコンソールへのログ出力に依存しており、通信処理が高速に進むため逐次的な状態遷移を追跡することが難しい。これにより、複雑な動作を含むプロトコルに対しては、異常動作の原因を把握しづらい。

これらの課題を解決するために、本研究ではデバッグ効率の向上に焦点を当て、実機でのプロトコル検証をより効果的に行える環境を提案する。なお、状態遷移図からのコード生成部については、[?] で改善が図られているため、そちらを参照されたい（たびたびそちらの論文を参照するように指示することがある）。

第 3 章

提案システムの概要と設計方針

本研究では、先行研究システムにおいて課題となっていたデバッグ効率の低さを改善することを目的として、新たなデバッグ支援システムを提案する。本章では、提案システムの概要と設計方針について述べる。はじめに、本研究で対象とする課題を整理し、次に設計方針およびシステム全体の構成について説明する。

3.1 設計方針

先行研究システムでは、状態遷移図を用いて通信プロトコルを設計し、実機上で動作検証を行うことが可能であった。一方で、デバッグ方法は主に図 3.1 のように、コンソールへのログ出力に依存しており、状態遷移の流れや内部動作を逐次的に把握することが難しいという課題があった。

本研究では、この課題を踏まえ、実機上で実行される通信プロトコルの内部動作をより直感的に把握できるデバッグ支援環境の構築を設計方針とした。具体的には、プロトコルの動作を状態遷移として捉え、その実行状況をログとして取得・可視化することで、動作の流れを段階的に確認できる仕組みを目指す。

また、本研究のシステムは、先行研究で構築されたプロトコル設計および実機検証の枠組みを活用することを前提とし、既存システムとの親和性を保ちながら改善を行うことを重視した。これにより、研究室内での継続的な利用や拡張が容易な構成とすることを設計上の方針とした。

```
STATE:IDLE
current_slot = 0
STATE:RECEIVE_SLOT_INFO
STATE:CHECK_RECEIVED_PACKET
STATE:WAIT_OWN_SLOT
current_slot = 1
STATE:WAIT_OWN_SLOT
STATE:WAIT_OWN_SLOT
current_slot = 2
STATE:WAIT_RANDOM_DELAY
STATE:CREATE_PACKET
STATE:TRANSMIT
.
.
.
```

図 3.1: 従来のデバッグ方法

3.2 システムの構成

システム全体の構成を図 3.2 に示す。

提案システムは、先行研究システムに対してデバッグ支援機能を付加する形で構成されている。全体としては、通信プロトコルを実行する組込みシステムと、その動作を外部から確認するためのデバッグ支援システムからなる。

組込みシステム側では、状態遷移に基づいて通信プロトコルが実行され、その実行過程に関する情報がログとして出力される。一方、デバッグ支援システム側では、出力されたログを収集し、状態遷移の流れや実行状況を可視化する役割を担う。

3.3 各コンポーネントの概要

提案システムは、主に以下のコンポーネントから構成されている。

- 状態遷移図からコードを自動生成するコンポーネント

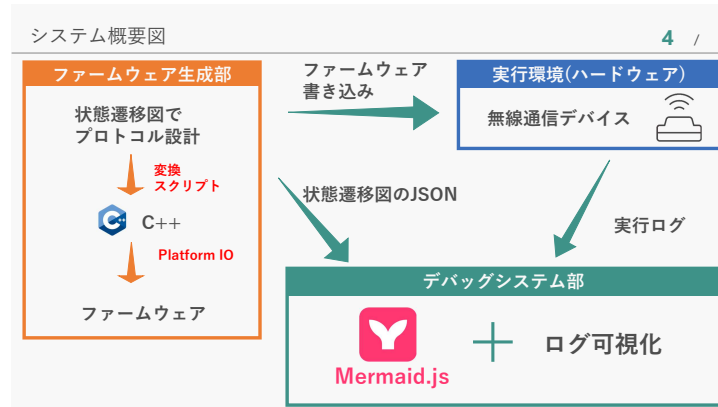


図 3.2: 提案システムの概要図

- 通信プロトコルを実行する組込みシステム
- デバッグ支援システム

3.3.1 コード生成コンポーネント

コード生成コンポーネントは、UMLの状態遷移図を解析し、通信プロトコルの動作を表現するプログラムコードを自動生成する役割を担う。本研究では、このコンポーネントとして共同研究（小林侑生ら）により開発された手法を利用している。本コンポーネントでは、状態遷移図の構造を解析し、存在する状態、遷移、イベントおよびアクションを抽出する。抽出した情報に基づき、通信プロトコルの動作を表現するC++コードを生成し、状態遷移はswitch-case文を用いて実装される。生成されたコードはPlatformIOを用いてビルドされ、無線通信デバイスに書き込まれた後、組込みシステム上で実行される。なお、本研究の主眼はコード生成手法そのものではなく、生成されたコードを用いた実機デバッグ支援環境の構築にあるため、コード生成手法の詳細については文献[?]を参照されたい。

3.3.2 組込みシステムコンポーネント

本システムにおける実行環境を図3.3に示す。本研究では、SAMD21マイコンと無線通信モジュールを一体化した無線通信デバイス（株式会社サーキットデザイン製どこでモ

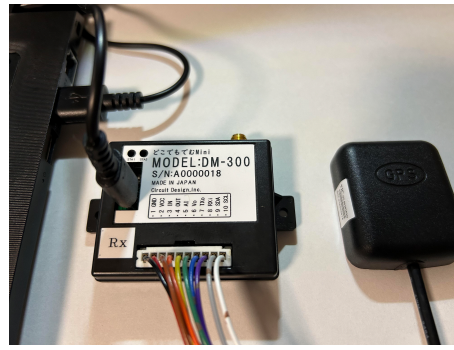


図 3.3: どこでモデム (左) と GPS モジュール (右)

デム) に GPS モジュールを組み合わせて、一つの無線通信デバイスとして使用した。この無線通信モジュールは 429MHz 帯で動作する汎用的なものであり、技術基準適合証明 (技適) を取得しているため、実際に電波を飛ばしながら通信プロトコルの検証を行うことができる。また、429MHz 帯は中山間地域において回折性能に優れており、低消費電力で長距離通信が可能な LPWA (Low Power Wide Area) 通信に適している。この特性を活かすことで、登山者見守りシステムなどへの応用も期待できる。さらに、本デバイスはマイコンと通信モジュールが一体化しているため、外部配線や追加ハードウェアを必要とせず、開発や実験環境の構築を容易に行うことが可能である。加えて、GPS モジュールを併用することで、位置情報の取得および 1PPS 信号による高精度な時刻同期を実現し、複数ノード間での時刻同期による無線通信プロトコルの実現を可能としている。GPS モジュールは使わない場合は外すこともできる。

3.3.3 デバッグ支援システムコンポーネント

最後に、本研究の中核であるデバッグ支援システムである。このコンポーネントは、組み込みシステムから出力されるログを収集し、状態遷移や内部動作を可視化する役割を担う。これにより、従来は把握が困難であったプロトコルの動作過程を確認できる。このコンポーネントは、次章で詳述するためここでは割愛する。

第 4 章

設計及び実装

本章では、提案システムの設計および実装について述べる。特に、デバッグ支援システムの設計方針、ログ取得方式、および状態遷移可視化の手法について詳述する。

4.1 使用ソフトウェア、ツール

使用ソフトウェア、ツールについて説明する。

本研究では、UML 設計ツールとして astah Professional を使用し、状態遷移図（ステートマシン図）の作成を行った。状態遷移図を用いることで、組込みシステムにおける複雑な状態変化やイベント駆動の挙動を視覚的に整理でき、設計段階での仕様理解や設計ミスの低減が可能となる。本研究における状態遷移図の基本的な設計方針や有効性については、[5] に詳しく述べられているため、そちらを参照されたい。

コード編集および開発環境としては Visual Studio Code を使用した。Visual Studio Code は軽量かつ拡張性に優れた統合開発環境であり、C++ や JavaScript など複数言語を横断的に扱う本研究の開発に適している。また、拡張機能を利用することで、構文ハイライト、コード補完、デバッグ支援などの機能を容易に追加でき、効率的な開発が可能である。

組込みシステムに書き込むファームウェアのビルド及び書き込みには PlatformIO を用いた。PlatformIO は、組込み向けの統合ビルド環境であり、ボード依存の設定管理やライブラリ管理を一元的に行えるという特徴を持つ。これにより、開発環境の再現性が高まり、異なる環境間でのビルド差異を抑えた開発が可能となる。

ブラウザ上で動作するデバッグ支援システムのフロントエンド開発においては、Node Package Manager (npm) を用いて必要なライブラリの管理を行った。npm を利用することで、JavaScript ライブラリや開発補助ツールを容易に導入・更新でき、依存関係の管理も効率的に行える。本研究では、これによりフロントエンド開発の作業負荷を軽減し、迅速な機能追加や修正を可能とした。

4.2 各コンポーネントの接続

各コンポーネント間の接続関係について説明する。はじめに、コード生成コンポーネントでは Python スクリプトを実行し、作成した astah の状態遷移図ファイルを入力として C++コードを生成する。この際、デバッグ支援システムで使用するための状態遷移図 JSON ファイルも同時に生成される。生成された C++コードは PlatformIO を用いてビルドされ、無線通信デバイスに書き込まれる。組み込みシステム上で実行された通信プロトコルは、動作中の状態遷移や内部変数の情報をログとしてシリアル通信を通じて出力する。これらのログはデバッグ支援システムによって受信され、解析および可視化に利用される。また、コード生成コンポーネントによって生成された状態遷移図 JSON ファイルは、デバッグ支援システムにアップロードされ、状態遷移の可視化に用いられる。

4.3 デバッグシステムの設計

本研究で提案するデバッグ支援システムは、ブラウザ上で動作するフロントエンドアプリケーションとして実装されている。

UI およびロジックの構築には React.js (以下 React) を採用した。React はコンポーネント指向に基づくフロントエンドフレームワークであり、画面要素を状態と結び付けて管理できるという特徴を持つ。本研究で扱うデバッグ支援システムでは、状態遷移の進行に応じて表示内容が頻繁に変化するため、状態変化に伴う UI 更新を効率的に行える React は適している。また、状態遷移図のハイライト表示や変数値の動的更新などを宣言的に記述できる点も、本システムの可読性および保守性の向上に寄与している。

UI のスタイリングおよびレイアウト設計には Bootstrap を使用した。Bootstrap は、あらかじめ定義された CSS クラスおよび UI コンポーネントを提供するフロントエンド

フレームワークであり、レスポンスな画面設計を容易に実現できるという特徴を持つ。本研究のデバッグ支援システムでは、多数の情報（状態遷移図、ログ、変数値など）を同一画面上に整理して表示する必要があるため、グリッドシステムやカードコンポーネントを活用することで、視認性と操作性の両立を図った。また、スタイル定義を最小限に抑えつつ統一感のある UI を構築できる点は、開発効率の向上および保守性の確保に寄与している。

4.3.1 動作モード

本システムは、単一ノードのログを解析する**ログモード**と、2台のノードを同期させて解析する**同期モード**の2つの動作モードを備えている。

ログモードでは、ユーザが状態遷移図の JSON ファイルをアップロードし、組込みシステムから送信されるログをリアルタイムで受信・解析する。解析されたログは状態遷移および各ステップにおける変数値として整理され、状態遷移図上に現在の状態がハイライト表示される。また、本システムではステップ実行機能を備えており、ユーザは状態遷移を一つずつ確認することができる。さらに、複数の変数を同時に監視できるウォッチ機能や、スライダー操作による任意ステップへのジャンプ機能も実装している。

同期モードでは、デバッグ支援システム上のシリアルモニタ機能を用いて、2台の無線通信デバイスからのログを同時に受信する。受信したログを基に同期タイムラインを生成し、両デバイスの状態遷移および変数値を同一の時間軸上で比較することが可能である。デバイスの実行ログの入力方法がログモードと異なるが、ログモードと同様に変数のウォッチ機能やステップ実行機能は共通である。

4.4 ログ設計

4.4.1 ログ生成の設計

ここでは、組込みシステムから生成されるログの設計について述べる。ログは、ソースコード 4.1 に示すように、状態遷移イベント、変数変更イベント及び、その他のログ（）で構成されている。これらの情報を基にデバッグ支援システムで要素の解析が行われ、その結果を可視化している。

状態遷移イベントは、組込みシステムの実行中に状態が遷移した際に生成されるログであり、STATE: 状態名 の形式で表現される。状態が切り替わるたびに組み込みシステムのファームウェア内の `updateState()` 関数が呼び出され、現在の状態をログ出力するという設計にした。これにより、デバッグ支援システムは受信したログから現在の状態を把握し、状態遷移図上で該当する状態をハイライト表示することができる。

変数変更イベントは、組込みシステム内の変数が変更された際に生成されるログであり、[VAR:<name>=<value>] の形式で表現される。変数の出力には C++ のマクロ機構を用いており、通常は `Serial.println("変数名": 値)` のように出力したい変数のそれぞれに対して、変数名を文字列として明示的に記述する必要があるが、本研究では C++ のマクロを用いてユーザの負担を簡略化している。本研究で定義したマクロは、変数からその名前と値を自動的に取得し、統一された形式でログを出力するため、開発者は出力形式を意識することなく、状態遷移図における各状態の entry 内に `PRINT_VAR_LOG(変数)` と記述するだけでよい。これにより、デバッグ用コードの記述量を抑えつつ出力形式の統一を実現し、デバッグ支援システムはこれらのログを解析することで、各ステップにおける変数値を追跡し、ウォッチ機能を通じてユーザに提供することが可能となる。

その他のログは、デバッグ情報やエラーメッセージなど、状態遷移イベントおよび変数変更イベント以外の情報を含む。これらのログは、実行状況の把握やトラブルシューティングの補助として利用されるが、状態遷移や変数値の解析対象とはしておらず、解析の際には無視される。

ソースコード 4.1: 生成されるログの例

```
1  STATE: LISTEN
2  [VAR: var01=0]
3  [VAR: var02=10]
4  STATE: LISTEN
5  STATE: RECEIVE
6  [VAR: var01=1]
7  [VAR: var02=11]
8  STATE: TRANSMIT
9  [VAR: var01=2]
```

10

[VAR: var02=100]

4.5 ログ解析の設計

本システムにおけるログ解析の設計について説明する。ログ解析は、デバッグ支援システムの中心的な機能であり、組込みシステムから送信されるログを受信・解析し、状態遷移および変数変更の情報を抽出する役割を担う。解析方法はそれぞれのモードによって異なる。

4.5.1 ログモードにおけるログ解析

ログモードでは、ユーザによりアップロードされた実行ログファイルを解析する。解析手順は以下の通りである。

- (1) ログファイルを行単位で読み込み
- (2) 各行が状態ログ、変数ログ、その他のログを判別
- (3) 状態ログおよび変数ログを抽出・整理

ログ行の分類

各ログ行は、正規表現を用いて先頭の文字列に基づき、以下の三種類に分類される。

- 状態遷移ログ： STATE: で始まる行
- 変数変更ログ： [VAR: で始まる行
- その他のログ：上記のいずれにも該当しない行

状態遷移ログおよび変数変更ログのみが解析対象であり、デバッグメッセージやエラーメッセージなどのその他のログは解析時には無視される。

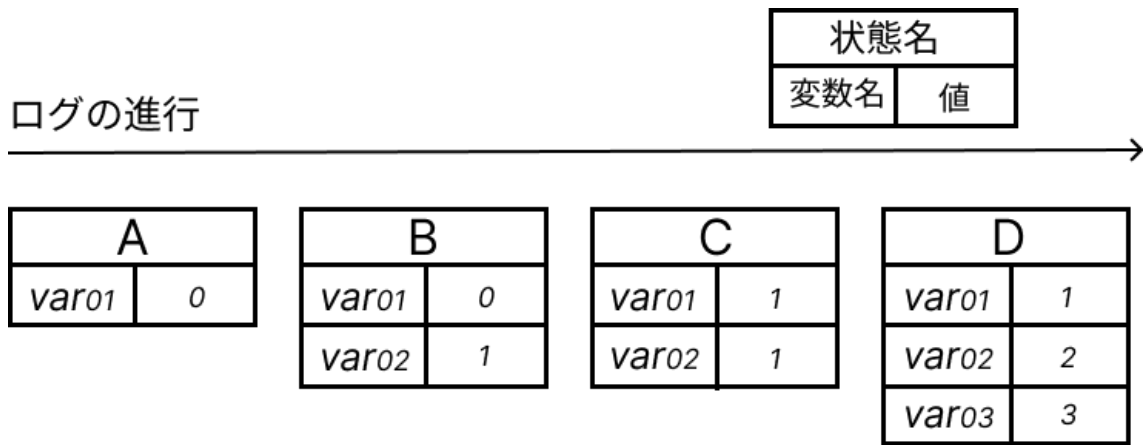


図 4.1: 変数更新ロジック

状態遷移ログの解析

状態遷移ログが検出された場合、ログ行から状態名を抽出し、新たな状態として記録する。同時に、その状態における内部変数の値を保持するため、直前までに観測された変数値集合を状態スナップショットとして保存する。本システムでは、状態ごとの完全な変数一覧が常にログとして出力されるわけではないため、状態遷移時点における変数値は、それまでに更新された最新の値を引き継ぐものとして扱う。

変数変更ログの解析

変数変更ログが検出された場合、ログ行から変数名および値を抽出し、内部で保持している変数集合を更新する。更新された値は、次に状態遷移ログが現れた際に、その状態に対応する変数値として反映される。なお、変数変更ログは「値が変更された場合」にのみ出力されるため、ログに現れない変数については、直前の状態における値が保持される。

図 4.1 に、変数更新のロジックを示す。状態遷移ログが現れた際に、その状態に対応する変数値は、それまでに観測された最新の値を引き継ぐ形で保存される。一方、変数変更ログが現れた際には、該当する変数の値のみが更新される。このようにして、各状態における変数値のスナップショットが構築される。

また、このロジックの擬似コードを以下に示す。

ソースコード 4.2: ログ解析アルゴリズム

```
1 states ← 空の配列
2 vars ← 空の配列
3 lastVars ← 空の連想配列
4
5 for 各ログ行 do
6   if 行が STATE ログ then 状態名を抽出
7
8     states に追加
9     vars に lastVars のコピーを追加
10  else if 行が VAR ログ then 変数名と値を抽出
11
12    lastVars を更新直前の状態スナップショット内の変数を更新
13
14  end if
15 end for
16
17 return (states, vars)
```

4.5.2 同期モードにおけるログ解析

同期モードでは、〇〇で後述するシリアルモニタからリアルタイムで受信したログを解析する。解析手順は以下の通りである。

4.6 ログモードの設計

本研究では、単一の無線通信デバイスの動作を詳細に解析・可視化するために、「ログモード」を設計・実装した。ログモードでは、組み込みシステムから送信されるログをリアルタイムで受信し、状態遷移および変数変更の情報を抽出・整理することを目的としている。

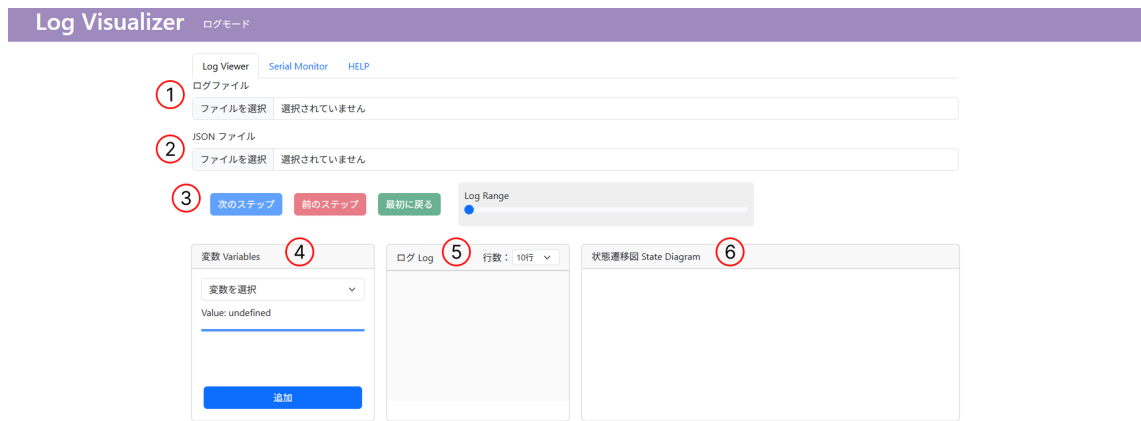


図 4.2: ログモードのレイアウト

4.6.1 ログモードのUI設計

ログモードのUIは、図4.2に示すように、主に以下の要素から構成されている。

- (1) 実行ログファイルをアップロードするためのファイルアップローダー
- (2) 状態遷移図の構造を持つJSONファイルをアップロードするためのファイルアップローダー
- (3) ステップ実行制御ボタン
- (4) スライダーによるステップジャンプコントロール
- (5) 変数のウォッチリスト表示エリア
- (6) 状態遷移ログ表示エリア
- (7) 状態遷移図表示エリア

4.7 同期モードの設計

本研究では、複数の無線通信デバイス間における挙動を比較・解析するために、2台の無線通信デバイスを同時に扱う「同期モード」を設計・実装した。同期モードでは、各デバイスから出力されるログを時系列で統合し、同一時刻における各デバイスの状態を可視化することを目的としている。

具体的には、2 台の無線通信デバイスを PC に接続し、それぞれ独立したシリアルポートを介してログを受信する。受信したログはデバイスごとに管理され、状態遷移イベントおよび変数変更イベントを解析対象とする。各イベントには受信時刻に基づくタイムスタンプを付与し、両デバイスのログを時系列順に統合することで、同期タイムラインを生成する。

この同期タイムラインにより、「ある時刻において各デバイスがどの状態にあり、どのような変数値を保持していたか」を一目で把握することが可能となる。

4.7.1 ブラウザ上でのシリアルモニタ実装

同期モードでは、専用のデスクトップアプリケーションを用いることなく、ブラウザ上でシリアル通信を扱えるように設計している。具体的には、Web Serial API を用いることで、Web ブラウザ上から直接シリアルポートに接続し、無線通信デバイスとの通信を実現した。

ユーザはブラウザ上の UI から接続するシリアルポートを選択でき、open ボタンおよび close ボタンを用いて、シリアルポートの接続および切断を制御する。接続中は、各デバイスから送信されるログをリアルタイムに受信し、シリアルモニタ上に逐次表示することが可能である。

この方式により、環境依存の少ないデバッグ環境を構築でき、複数デバイスを同時に扱う同期デバッグを容易に実現している。

4.7.2 同期タイムラインの生成

同期タイムライン生成処理では、各デバイスから受信したログエントリに対して、受信時刻を基準としたタイムスタンプを付与する。状態遷移ログおよび変数変更ログは、それぞれイベントとして扱われ、デバイスごとに時系列順に管理される。

その後、複数デバイスのログを時刻情報に基づいてマージし、全デバイスに共通の時系列イベント列を構築する。この統合されたイベント列から、各時刻におけるデバイスの状態スナップショットを再構成し、同期タイムラインとして可視化する。

この同期タイムラインにより、複数デバイス間の相互作用や通信タイミングのずれを直感的に把握することが可能となり、分散システムのデバッグ効率を大幅に向上させる

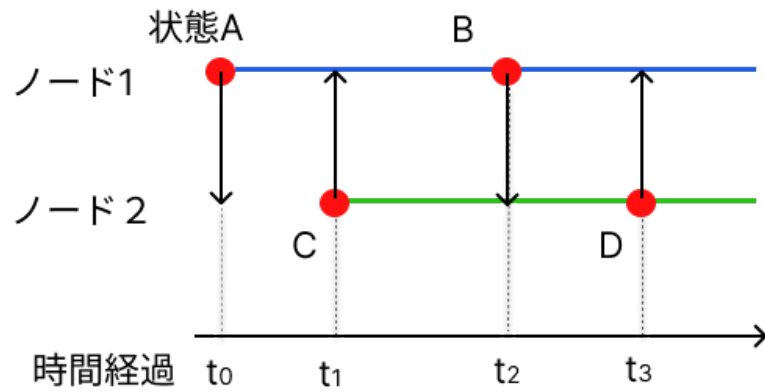


図 4.3: 同期モードのタイムライン

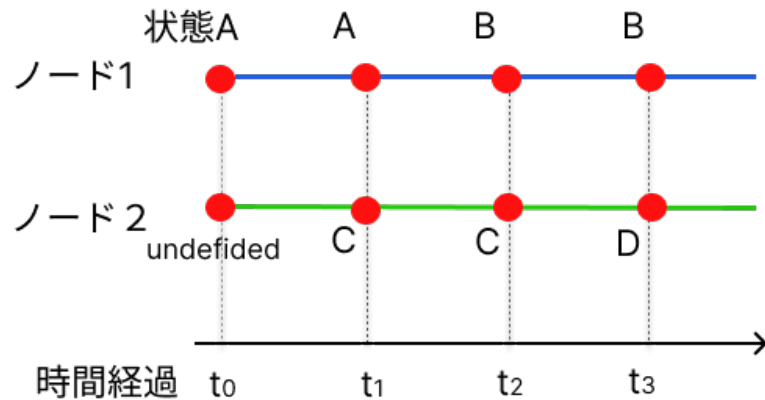


図 4.4: 同期モードのタイムライン

ことができる。4.3 および 4.4 に、実際の同期モードにおけるタイムライン表示例を示す。

4.8 状態遷移可視化の設計

状態遷移の可視化には、astah で作成した状態遷移図を JSON 形式でエクスポートしたものを使用する。ユーザはデバッグ支援システム上で JSON ファイルをアップロードし、Mermaid.js を用いて状態遷移図を描画する。組込みシステムから取得したログと連携することで、状態遷移図上に現在の状態をハイライト表示し、状態遷移の進行を視覚的に確認できる。

4.9 実装概要

実装概要を書く。

第 5 章

操作例

5.1 操作例の概要

本章では、提案システムの具体的な操作手順を示し、無線通信プロトコルのデバッグ作業においてどのような情報が得られるかを明らかにする。対象とする利用シナリオは、実機上で動作する通信プロトコルの挙動確認および不具合解析であり、従来のコンソールログベースのデバッグ手法と比較して、提案システムによって得られる利点を示すことを目的とする。特に、本章では教育的観点から、状態遷移の可視化およびステップ実行機能の有効性に着目する。

5.2 操作対象および前提条件

操作例では、状態遷移図に基づいて生成された MAC プロトコルを対象とする。本研究では、教育的に理解しやすい基本的な通信方式である pure ALOHA 方式をベースとした簡易的な MAC プロトコルを用いる。pure ALOHA は状態数および遷移が比較的少なく、状態遷移に基づく動作理解が容易であるため、本研究におけるデバッグ支援機能の説明に適している。

本プロトコルの状態遷移図は UML 設計ツールで作成し、JSON 形式で出力したものをを用いる。無線通信デバイスは 2 台構成とし、それぞれを独立したノードとして動作させる。各ノードからは、状態遷移および内部変数の変化を含むログがシリアル通信を通じて出力される。

以下では、まず単一ノードを対象としたログモードでの操作例を示し、続いて複数ノードを対象とした同期モードでの操作例について説明する。

5.3 状態遷移図の読み込み

はじめに、デバッグ支援システムを起動し、対象とする状態遷移図の JSON ファイルをアップロードする。アップロードされた状態遷移図は Mermaid.js を用いて描画され、状態および遷移の関係を視覚的に確認できる。これにより、プロトコル全体の構造を直感的に把握することが可能となる。

5.4 ログ取得と状態遷移の可視化

次に、無線通信デバイスを起動し、シリアル通信を通じてログを取得する。取得されたログはデバッグ支援システム内部で解析され、状態遷移および各ステップにおける変数値として整理される。解析結果は状態遷移図上に反映され、現在の状態がハイライト表示される。ログの進行に応じてハイライト表示が更新されるため、状態遷移の流れを視覚的に追跡できる。

5.5 ステップ実行による逐次確認

提案システムでは、取得したログをステップ単位で再生する機能を備えている。ユーザは操作ボタンを用いて状態遷移を一つずつ進めたり、前の状態に戻したりすることができる。また、スライダー操作により任意のステップへ直接移動することも可能である。これにより、通信処理が高速に進行する場合であっても、状態遷移を逐次的に確認できる。

5.6 変数ウォッチ機能の利用

状態遷移の確認に加え、ユーザは特定の内部変数を選択して監視することができる。複数の変数を同時に表示することが可能であり、状態遷移と変数値の変化の関係を同時に把握できる。これにより、プロトコル内部の動作をより詳細に確認できる。

5.7 複数ノード同期デバッグ

次に、複数ノードを対象とした同期モードでの操作例を示す。同期モードでは、デバッグ支援システム上のシリアルモニタ機能を用いて、2台の無線通信デバイスからのログを同時に取得する。取得したログを基に、デバッグ支援システム内で同期タイムラインを生成し、両ノードの状態遷移および変数値を同一の時間軸上で比較する。なお、本同期処理はGPS等による時刻同期を用いず、ログの取得順序および内部タイムスタンプに基づいて行われる。

5.8 操作例のまとめ

以上の操作例より、提案システムを用いることで、状態遷移の流れを視覚的に把握できること、ステップ実行によって逐次的な確認が可能であること、および複数ノード間の動作関係を同時に確認できることを示した。

第 6 章

評価と考察

6.1 評価方針

本章では，提案システムの有効性を定性的に評価することを目的とする。評価は，従来のコンソールログベースのデバッグ手法と比較することで行い，特にデバッグ効率および状態遷移の理解容易性の観点から検討する。

6.2 評価方法

評価対象として，同一の通信プロトコルおよび同一の不具合を用意し，従来手法と提案手法の 2 通りでデバッグを実施した。本研究では，実際のデバッグ作業を通じて得られた知見を基に，操作性や理解のしやすさといった観点から定性的な比較を行う。

6.3 デバッグ効率の評価

従来手法では，ログが高速に出力されるため，状態遷移の対応関係を把握するためにログを何度も確認する必要があった。一方，提案手法では，状態遷移の可視化およびステップ実行機能により，動作を逐次的に確認することが可能となり，不具合の原因を把握するまでの作業負荷が軽減された。

6.4 状態遷移理解の容易性に関する評価

従来手法では、状態遷移の流れをログから読み取り、利用者が頭の中で再構成する必要があった。これに対し、提案手法では現在の状態が常に状態遷移図上に表示されるため、状態遷移の流れを直感的に把握できた。

6.5 複数ノード同期デバッグの評価

複数ノードを用いた場合、従来手法ではノードごとのログを個別に確認し、手動で突き合わせる必要があった。提案手法では、同期タイムライン上で両ノードの状態遷移を同時に確認できるため、ノード間の状態ずれや動作の不整合を容易に把握できた。

6.6 考察

以上の評価結果より、提案システムは、複雑な状態遷移を含む通信プロトコルや、複数ノードが関与する通信において特に有効であることが示された。一方で、本評価は定性的な観点に基づくものであり、今後は定量的指標を導入した評価手法の検討が課題として挙げられる。

6.7 評価のまとめ

本章では、提案システムがデバッグ効率の向上および状態遷移理解の容易化に有効であることを定性的に示した。

第 7 章

まとめと今後の展望

7.1 まとめ

本研究の成果と意義を総括する。本研究の新規性は以下の 3 点にある。1. 実機ログを用いた状態遷移の逐次可視化 2. ステップ実行可能なデバッグ支援 3. 複数ノードの同期タイムラインによる比較分析

7.2 今後の展望

今後の課題と研究の方向性について述べる。

参考文献

- [1] IoT Analytics: "State of IoT 2024: Number of Connected IoT Devices Grows 16% to 16.6 Billion", 2024.
- [2] Grand View Research: "Industrial Wireless Sensor Network Market Size, Share & Trends Analysis Report, 2024–2030", 2024.
- [3] 旭 健汰, アサノ デービッド, 不破 泰: "無線モデムを用いたセンサネットワーク MAC プロトコル検証システムの開発", 信学技報, NS2023-147, pp.120–125, Dec. 2023.
- [4] 小林 遼, アサノ デービッド, 不破 泰: "センサーネットワーク検証システムにおける遷移図を用いた MAC プロトコル設計環境の開発", 信学技報, NS2023-148, pp.126–131, Dec. 2023.
- [5] 小林 侑生, アサノ デービッド, 不破 泰:
- [6] PlatformIO Labs, "What is PlatformIO?", <https://docs.platformio.org/en/latest/what-is-platformio.html>, 参照 Oct. 15, 2025.
- [7] Mermaid.js, "Overview," <https://mermaid.js.org/intro/>, 参照 Oct. 15, 2025.
- [8] Mermaid.js, "State Diagram Syntax (stateDiagram-v2)," <https://mermaid.js.org/syntax/stateDiagram.html>, 参照 Oct. 15, 2025.
- [9] Eclipse Foundation, "MQTT: Message Queuing Telemetry Transport," <https://mqtt.org/>, 参照 Oct. 15, 2025.
- [10] 園田 継一郎, 不破 泰, アサノ デービッド, "無線センサーネットワークの端末・中継機における送信タイミング決定時間短縮方法の検討," 信学技報, NS2022-138, Dec. 2022.